LaserLearn: A Real-Time Green-Laser Vision Platform for Projected Games on Commodity Hardware

Tonmoy Bishwas
Dept. of Computer Science and Engineering
United International University
ID: 0152410030, Dhaka, Bangladesh
tbishwas2410030@bsds.uiu.ac.bd

Samiha Mahzabin
Dept. of Computer Science and Engineering
United International University
ID: 0152410071, Dhaka, Bangladesh
smahzabin2410071@bsds.uiu.ac.bd

Naimur Rahman
Dept. of Computer Science and Engineering
United International University
ID: 0152410029, Dhaka, Bangladesh
nrahman2410029@bsds.uiu.ac.bd

Asif Hasan Shanto
Dept. of Computer Science and Engineering
United International University
ID: 0152410035, Dhaka, Bangladesh
ashanto2410035@bsds.uiu.ac.bd

Lailatul Noor
Dept. of Computer Science and Engineering
United International University
ID: 0152410041, Dhaka, Bangladesh
lnoor2410041@bsds.uiu.ac.bd

***Abstract*—We present LaserLearn, a real-time platform that turns a handheld green laser pointer into a game controller for a projected display, and the two games we built on top of it. A phone camera films the projection; a single-class YOLO detector, fine-tuned on our own green-laser dataset, finds the laser dot in each frame (test mAP50 of 0.88 at full resolution); and a one-time four-corner homography maps the dot into game space. Everything below the detector is deliberately small: games receive the laser through one narrow input object that exposes only what a real laser can do, which is what let us build a second, quite different game without touching the vision code. The first game hides a thief in the dark and uses the laser as a flashlight; the second is a laser-slicing arcade game. For the thief's behaviour we built and compare two adversaries: a tabular Q-learning policy trained against scripted laser opponents (95 percent win rate in simulation), and an A* pathfinding controller that plays on procedurally generated mazes. The harder problem was not accuracy but speed on the demonstration machine, a laptop with no discrete GPU. Our main practical finding is that on that machine real-time feel was limited by contention for the CPU between the inference thread and the rendering loop rather than by raw inference latency; moving inference onto the integrated GPU through OpenVINO removed the contention and gave the largest single improvement we measured. We close with the further games the same platform is meant to host.**

***Index Terms*—laser-pointer interaction, object detection, YOLO, real-time inference, edge inference, integrated-GPU offload, projector-camera systems, spatial augmented reality, reinforcement learning, pathfinding**

I. Introduction

Most students meet computer vision and reinforcement learning through a screen. A detector draws boxes on a recorded clip; an agent plays inside a simulator. The techniques work, but they stay behind glass. We wanted to pull them out into a single physical loop that a person can stand in front of and play, and to do it with hardware a student actually owns.

LaserLearn is that loop. A projector throws the game onto a wall. A phone camera on a tripod watches the wall and streams to a laptop. The player points an ordinary green laser at the projection, and in every camera frame a fine-tuned detector locates the dot and a homography carries its position into the game. What the player does with that dot depends on the game. In our first game the screen is almost entirely black and the laser is a flashlight: it reveals a small circle of the world, and somewhere in the dark a thief is creeping through a maze toward a treasure. Hold the light on the thief long enough and you catch it; let it reach the treasure and it wins. Our second game uses the same laser as a blade, and the player slices fruit that is thrown up across the screen.

Building two games this different on one pipeline was a deliberate goal, not an afterthought. Underneath both sits the same chain—camera, detector, homography, and a tiny input layer—and the games never see anything below that layer. They receive one object each frame that reports where the laser is, whether it is currently on the surface, and how fast it is moving. Nothing else. Because that object is all a real laser can honestly provide, a game written against it cannot accidentally depend on a mouse click or a key press, and a new game is mostly a matter of deciding what to do with a moving point. The second game took far less work than the first for exactly this reason.

This paper reports three things. First, the platform and its detection front end—a green-laser detector we trained ourselves, the calibration that places its output on the screen, and the input abstraction that the games sit on. Second, the two games, including two different controllers we built for the thief and what we learned by comparing them: a tabular Q-learning policy that learns to evade in a fixed maze, and an A* controller that plans on a new random maze every round. Third, and the part we found most interesting, the engineering

that made the whole thing feel instant on a GPU-less laptop.

That last point shaped the project more than any algorithm did. Our development desktop has a discrete GPU and runs everything without complaint. The machine we had to demonstrate on does not: it is a consumer laptop with an Intel processor and integrated graphics. Closing that gap turned up a result we did not expect. On the CPU-only target, the inference thread and the rendering loop were fighting over the same processor, and the system felt sluggish even though each task on its own was fast enough. Raw inference speed was a red herring. Moving inference onto the integrated GPU through OpenVINO took it off the CPU entirely, ended the contention, and was the biggest speed-up we got.

Contributions.

- A real-time, multithreaded laser-projection platform built around a fine-tuned single-class green-laser detector and a four-corner homography, with a narrow laser-faithful input layer that lets new games be added without touching the vision pipeline.
- Two games on that platform—a flashlight-reveal thief hunt and a laser-slicing arcade game—and, for the thief, two adversary controllers we built and compare: a trained tabular Q-learning policy and an A* planner.
- A deployment study on commodity, GPU-less hardware whose central finding is that CPU contention between inference and rendering, not inference speed, governs real-time feel, and that integrated-GPU offload resolves it.

The rest of the paper is organised as follows. Section II reviews related work. Section III describes the platform and its input layer. Section IV gives the methods: detection, calibration, the flashlight reveal, and the two thief controllers. Section V describes the two games and the ones we plan next. Section VI covers the implementation and deployment problems, Section VII reports results, Section VIII discusses limitations, and Section IX concludes.

II. Related Work

LaserLearn sits at the meeting point of several lines of work. We take each in turn and end every part by saying how our system departs from it. Table I summarises the comparison.

A. Laser-Pointer Interaction and Laser-Dot Detection

Tracking a laser pointer on a projected display with a camera is an old idea. Kirstein and Müller showed a camera-tracked laser-pointer-as-mouse for projection screens [1], and Olsen and Nielsen's "Laser Pointer Interaction" [2] is the standard reference: a camera looks for the brightest red spot on the surface, a calibration step maps camera coordinates to display coordinates, and the interaction copes with the laser's jitter and with the lack of a real click. Later work pushed the idea to large and tiled displays [3] and studied detection on its own. Lapointe and Godin argued that reliable spot detection is the limiting factor for a usable laser-pointer interface and built a fast classical detector [4]. For harder, less constrained scenes the detectors get more elaborate: Krstińić et al. combine a modified circular Hough transform with optical-flow motion analysis to pull the true dot out of competing bright blobs under a moving camera [5], and Chávez et al. mix template matching with a fuzzy rule base to find a laser spot in real homes [6]. The shared assumption, and the shared weakness, is that the dot is the brightest or reddest thing in view—which fails under projector glare and coloured, moving content.

How LaserLearn differs. We drop the brightness-and-colour heuristic for a single-class detector fine-tuned on the dot's learned appearance, which holds up better against glare and against the animated content of a real game. Our camera is a handheld phone, not a fixed calibrated rig, and the laser is not a cursor but a spatial flashlight (and, in our second game, a blade)—uses this literature does not cover.

B. Projector-Camera Games and Spatial Augmented Reality

Turning ordinary surfaces into interactive displays with a projector and a camera is the premise of spatial augmented reality (SAR). Raskar et al. laid the groundwork with the Office of the Future [7], the framing of spatially augmented reality [8], and Shader Lamps, which projects image-based shading onto neutral objects [9]. Pinhanez's Everywhere Displays steered a projector onto arbitrary surfaces and read hand gestures with a camera, and was used to build projected games [10]. More recent projector-camera gaming systems such as IllumiRoom [11] and RoomAlive [12] extend play into a whole room using calibrated depth cameras and several projector units. Closer to our budget, Song et al. built a tangible game interface from a cheap projector-camera setup [13]. The camera-to-surface map is a homography; careful projector-camera calibration has been studied thoroughly—Moreno and Taubin note that one global homography cannot model projector lens distortion and use local homographies for sub-pixel accuracy [14], building on the standard treatment of planar homographies [15]. The flashlight-as-reveal idea also has a precedent: Ridel et al.'s Revealing Flashlight lets a user sweep a virtual light over a heritage artifact to bring out surface detail [16], itself a descendant of the movable Magic Lens of Bier et al. [17].

How LaserLearn differs. We use the most minimal procam setup we can—one consumer projector, one phone camera, one global four-corner homography—rather than the multi-unit depth rigs of IllumiRoom and RoomAlive, because our surface is flat and a coarse game grid does not need sub-pixel registration. That same simplification, in line with the distortion caveat of [14], also explains a small spotlight offset we see and treat as a known limitation (Section VIII). Unlike the Revealing Flashlight, our reveal is driven by a tracked physical laser rather than a tracked stylus, and it is the core mechanic of a live game, not a tool for inspecting a static object.

C. Real-Time Vision Deployment on Commodity Hardware

Our detector is from the YOLO family of single-shot real-time detectors [27], whose later versions and small-object behaviour are surveyed by Terven et al. [28]; a laser dot is

a very small object, a case the original YOLO already flags as hard. Running such models off the GPU is supported by toolkits like OpenVINO, whose analysis-and-tuning workbench targets Intel CPUs and integrated GPUs [29], and recent work benchmarks inference frameworks on edge hardware [30]. A separate line of systems work treats *resource contention* as a first-class problem when several models share one constrained device and schedules around it [31].

How LaserLearn differs. The benchmark literature mostly measures inference *in isolation*, and the contention literature looks at *multiple models* sharing a device. Our binding constraint is neither: contention between a *single* inference pipeline and an *interactive rendering loop* on a CPU-only consumer laptop. Our fix—pushing inference onto an Intel Iris Xe integrated GPU via OpenVINO [29] to free the CPU for rendering—turns that inference-versus-render trade-off from the dominant bottleneck into a non-issue, which an isolated benchmark would never reveal.

D. Adversary Behaviour: Learned and Search-Based

We built the thief's behaviour two ways, so we draw on two literatures. For the learned controller, Watkins and Dayan's Q-learning supplies the algorithm and a convergence guarantee in the tabular, finite, discrete case [18], and Sutton and Barto ground the MDP formulation, discounting, and exploration schedule we use [19]. Deep RL—epitomised by the Deep Q-Network that learns Atari from pixels [20]—is the tool of choice when the state is large or continuous, which ours is not. Pursuit and evasion are naturally multi-agent: Tan contrasts independent and cooperative Q-learners in a predator-prey grid [21], Littman frames zero-sum play as Markov games solved by minimax-Q [22], and more recent work learns pursuit policies with deep RL and transfers them to real drones [23]. For contrast, OpenAI's hide-and-seek shows competitive self-play producing an emergent curriculum of strategies [24], and RL has also been used to make game bots less predictable [25]. For the search-based controller we use A*, the classic best-first graph search with an admissible heuristic of Hart, Nilsson, and Raphael [26]; in our grid the natural heuristic is Manhattan distance, and the laser enters as an added traversal cost rather than as a co-learning agent.

How LaserLearn's adversaries differ. Our state for the learned thief is a compact four-tuple—thief cell and laser cell on a fifteen-by-nine grid—with four actions, so a Q-table is small, trains in seconds on a CPU, is fully readable, and inherits the convergence guarantee of [18]; deep RL as in [20], [23] would add instability and tuning pain for no gain at this size. Rather than the self-play autocurriculum of [24], we train against an *exogenous*, hand-designed set of scripted laser behaviours, following the independent-learner view of [21], which keeps training cheap and reproducible and aimed at the opponents the game can actually present. The A* controller takes a different stance entirely: it does no learning, replans every cell, and so handles a freshly generated maze each round, where the trained table is tied to the one maze it learned. Comparing the two on the same game is part of what we report (Section IV-D).

Research Gap

Every ingredient has precedent: camera-tracked laser interaction is decades old, procam games and SAR are mature, edge inference is active, and both tabular Q-learning and A* are textbook. What we put together is the combination: a *fine-tuned single-class laser detector* feeding a *homography-mapped input layer* that carries one physical laser into *more than one real-time projected game* on *commodity, GPU-less hardware*, with the deployment finding that processor contention—not inference speed—sets the real-time feel.

III. Platform and Input Layer

LaserLearn runs as a multithreaded pipeline whose stages are decoupled so that slow inference never stalls rendering. A *capture thread* reads only the newest frame from the phone camera (camera buffer size one, no queue). A *detector thread* runs the YOLO model on that frame, keeps the highest-confidence detection, and maps its box centre through the homography into game coordinates, publishing the result as the current laser position. A *rendering loop* reads whatever the latest laser position is and draws the frame. Because the detector only ever publishes the most recent position and the renderer never waits on it, the detection rate and the render rate float free of each other—which is exactly what makes the contention behaviour in Section VII both visible and tunable. A device *profile* fixes the inference backend, image size, confidence threshold, and camera index before the threads start, so the same code runs on the discrete-GPU desktop and the integrated-GPU laptop with a one-word change.

The input layer. Sitting on top of all of this is one small object, and it is the part of the design we lean on most. Every game receives, per frame, a pointer snapshot: an (x, y) position in the fixed 1920×1080 game canvas, a boolean saying whether the laser is on the surface this frame, and the velocity (v_x, v_y) since the last frame (from which a speed is derived). That is the entire contract. There is no mouse button, no key, no scroll, no second cursor, because a laser pointer offers none of those. During development we feed this same object from the mouse so the games can be built and played on a normal computer; at the installation a laser-backed source fills it instead, and the active flag simply follows whether the beam is currently being detected. The games cannot tell the difference, and neither can their code.

Two helpers turn that bare pointer into usable interaction without ever adding a click. A *dwell tracker* fills a progress value from zero to one while the pointer is held near a target and drains it (at half speed) when it drifts off; crossing one fires the action. This is how the laser "clicks"—to choose a game from the menu, and, in the thief game, to make a catch. A *gesture* helper asks whether the straight segment between the pointer's previous and current position passes within some radius of a target while moving above a speed threshold; that is how the laser "slices" in the second game. Both helpers read

TABLE I
REPRESENTATIVE PRIOR SYSTEMS VS. LASERLEARN

System	Input modality	Dot/target detection	Calibration	Adversary	Hardware target
Olsen & Nielsen [2]	Laser pointer (cursor)	Brightest-red-spot heuristic	Camera-to-display map	None	Desktop + camera
Pinhanez (Everywhere Displays) [10]	Hand gesture	Vision gesture detection	Oblique-projection correction	None	Steerable projector rig
RoomAlive [12]	Body / depth	Depth sensing	Self-calibrating procam units	Scripted	Multiple projector-depth units
Revealing Flashlight [16]	Tracked stylus	Tracked device pose	SAR projector calibration	None (inspection)	Workstation + projector
Hide-and-seek (OpenAI) [24]	Simulated agents	N/A (simulator)	N/A	Self-play (emergent)	Large-scale GPU cluster
LaserLearn (ours)	**Green laser pointer (flashlight + blade)**	**Fine-tuned single-class YOLO**	**Single 4-corner homography**	**Q-learning + A* thief**	**GPU-less laptop + iGPU**

only the pointer snapshot, so any game can use them, and a new game that needs "hover to select" or "swipe through" gets it for free. We come back to why this matters for adding games in Section V-C.

IV. METHODS

A. Laser Detection

The laser dot is found by a single-class object detector. We use a green 532 nm laser—bright enough to read on the projection under normal room light—and fine-tuned a nano-scale YOLO model (about 2.5 M parameters, the smallest of a recent YOLO release) on our own dataset, annotated with one class, `laser`. The dataset is our own: 1069 full-resolution (1920×1080) frames, with 675 laser annotations across the single class. Two choices in how we built it mattered. First, we shot the green dot against a wide range of background colours rather than one neutral wall, so the detector would not quietly learn "green on grey" and then fail when a game drew a coloured scene. Second, and more deliberately, 398 of the 1069 frames—about 37 percent—contain no laser at all. These negatives are not waste; they teach the model what *absence* looks like, so it stays silent when the dot is off the surface instead of inventing one. For a controller where a single false detection yanks the spotlight to the wrong place, suppressing false positives is at least as important as finding the true dot. We exported the set from Roboflow with a 70/20/10 split and applied augmentation to the training portion only, which expands it to 2168 training images against 214 validation and 107 test images held out unaugmented. Training ran on the development GPU with mixed precision. We use the nano model on purpose: it is small enough to run on the laptop's integrated GPU and even on its CPU, which is the whole point of the project.

At run time each camera frame goes to the detector; if it returns one or more boxes we take the highest-confidence one and use its centre as the laser's camera-pixel position. A modest confidence threshold suppresses spurious responses, and because we keep only the best box, the detector emits at most one laser position per processed frame for the renderer to consume asynchronously. We report accuracy at two input sizes in Section VII: the model's native size, and the smaller size we actually deploy at for speed.

B. Camera-to-Game Calibration

The detector speaks in camera pixels; the game thinks in its own 1920×1080 space. We bridge them with a one-time four-corner calibration. The operator clicks the four corners of the projected area in the camera image, in the order top-left, top-right, bottom-right, bottom-left; the matching game-space corners are the corners of the 1920×1080 rectangle. From these four correspondences we compute a 3×3 homography H (with `cv2.findHomography`) and map every detected point through it (with `cv2.perspectiveTransform`). The homography is saved and reused until the camera or projector moves. One global homography is enough here because the surface is flat and the game grid is coarse; the small residual error it leaves is discussed in Section VIII. A constant pixel offset, adjustable live with the arrow keys and saved with the calibration, soaks up any systematic bias between where the laser points and where the spotlight lands.

C. The Flashlight-Reveal Interaction

The first game's interaction is built on occlusion, not a cursor. Each frame is drawn normally and then covered by a near-opaque dark overlay; a soft-edged circular hole is punched in the overlay at the laser position, so only a small disc around the laser is visible. In practice the overlay is one alpha surface that we allocate once and reuse, and the spotlight is a pre-built radial texture subtracted into it at the laser point—details that matter for performance, as Section VII explains. The lit disc has a radius of 160 game pixels with a soft falloff.

Three small behaviours keep the light from feeling twitchy. It fades in over a fraction of a second when a detection arrives and fades out when the laser disappears, so it does not flicker on and off. A detection counts as current only

for a short staleness window; past that the light starts to fade. And an optional first-order filter on the mapped position trades a little latency for steadiness. We tuned all of these—radius, fade time, staleness, smoothing, falloff, darkness—in a stripped-down sandbox that runs the exact camera, detector, and calibration but draws only the reveal, so the settings carry straight over.

Edge-biased maze. While testing we noticed the detector was most reliable near the left and right of the frame and weakest in the centre, a bias we trace to the training data (Section VIII). Instead of only gathering more images, we turned it into a level-design rule: the maze the thief travels is laid out so its main corridors hug the edges, putting the action where detection is strongest. It is a design response to a model limitation, and it cost nothing to apply.

D. Two Thief Controllers

The thief has to feel like a real opponent: it should head for the treasure, notice when the light is on it, and try to get away. We built its behaviour two different ways over the life of the project, and since they make an instructive contrast we describe both.

Learned: tabular Q-learning. The first controller treats the thief as a small Markov decision process. The state is the pair of grid cells holding the thief and the laser, $(r_t, c_t, r_\ell, c_\ell)$, with $(-1, -1)$ for "no laser"; the actions are the four grid moves, and a move into a wall is a penalised no-op. The reward is shaped to want the treasure and fear the light: reaching the treasure ends the episode with $+1$, being held in the light long enough ends it with -1, a small -0.01 per step discourages dawdling, a -0.05 discourages bumping walls, and running out of time costs -0.5. Because the state and action spaces are small and discrete, we store the policy as a sparse Q-table and learn it with ordinary Q-learning ($\alpha = 0.1$, $\gamma = 0.95$, ε-greedy decaying from 1.0 to 0.05 over the first 80 percent of training), which is readable and converges in this finite case [18], [19]. To keep the thief from overfitting one kind of opponent, each training episode samples one of four scripted laser behaviours: an idle laser (so it just learns to navigate), a patrolling laser, a human-like laser that tracks the thief with reaction lag and jitter, and a random laser. We trained for 20,000 episodes on the desktop GPU, which takes only a few seconds and ends with about 3,900 learned states. At play time the table is loaded greedily, with no further learning. This controller is fast and interpretable, but it learns one specific maze; change the maze and it has to be retrained.

Search-based: A* with laser avoidance. The second controller plans instead of learning. Each round generates a fresh random maze, so a fixed policy is no use; the thief instead runs A* over the grid with a Manhattan-distance heuristic [26] and replans at every cell. The laser enters as a cost, not as a wall: cells near the current laser position carry an extra traversal penalty, so the planner naturally bends the route away from the light while still reaching its goal, and a fallback plan that ignores the laser keeps it from freezing if the light sits across the only corridor. The game is two-phase. First the thief heads for the treasure; once it has the treasure the goal switches to escaping through one of two doors placed far apart and far from the treasure, so a single spotlight cannot cover both exits. The thief scores the doors as it runs, adding a penalty when the laser is guarding a door or sitting on the route to it, and it commits to one only once it has left the shared part of the path—with a little hysteresis so it does not dither between them. Two reactive touches make it read as alive: when the light lands on it, it bolts, briefly cutting its cell-crossing time and quickening its walk animation, and the catch is a hold, filling while the light stays within 100 pixels of it and draining when it slips off, so the player must track it rather than tag it once.

What the comparison taught us. The two controllers trade off along clear lines. Q-learning gives genuinely emergent evasion—it discovers, for example, that hugging a wall away from a patrolling laser pays off—but only on the maze it trained on, and its quality is bounded by how well our four scripted lasers stand in for a real player. A* needs no training and drops onto any freshly generated maze, with avoidance that is explicit and easy to reason about, but its behaviour is only as clever as the costs we hand-write; it does not *learn* anything about how this particular player aims. The practical reason we ended up leaning on A* for the random-maze version is mundane but real: procedurally generated mazes broke the one assumption the trained table depended on. We think the honest reading is that the learned policy is the better fit for a fixed, hand-designed level, and the planner is the better fit once levels are generated on the fly—a point we expand in future work.

V. The Games

A. Game One: Thief Hunt

The first game puts everything above together. The player sees a black screen and a small moving pool of light wherever the laser points. Hidden in the dark, a thief works its way through the maze toward a treasure; the player sweeps the light to find it and then holds the light on it to catch it. Catching is the dwell mechanic from Section III: a ring fills while the light stays on the thief and drains when it comes off, and a full ring is a catch. The thief wins by reaching the treasure (in the random-maze version, by then escaping through a door). The treasure itself is hidden in the dark like everything else, which, as a side benefit, stopped the detector from ever mistaking a permanently lit object for the laser. The thief is driven by one of the two controllers of Section IV-D.

B. Game Two: Fruit Ninja

The second game reuses the same laser as a blade. Fruit is thrown up from below on gravity arcs, and the player slices it by sweeping the laser through it. A slice is the gesture from Section III: the segment between the laser's last and current position has to pass within a fruit's radius while the laser is moving above a speed threshold (we use 22 pixels per frame). The speed gate is what separates slicing from aiming—resting the laser on a fruit does nothing, so the player cannot cheat

by parking the dot. As the score climbs, fruit spawns faster and bombs start to appear; slicing a bomb, or letting fruit fall unsliced, costs a life, and several fruit caught in one fast sweep score a combo. This game deliberately turns the on-screen pointer marker off, because the laser dot is itself the visible blade and a second white marker drawn on top of it only washed out the green the detector was keying on. One laser-specific detail mattered more than we expected: when the beam blinks out for a frame, we throw away the last position, otherwise the next detection a moment later would draw a slice straight across every fruit in between.

The point of describing the two games side by side is how little the second one cost. Fruit Ninja shares the entire stack below the input layer with Thief Hunt—same camera, same detector, same calibration, same pointer snapshot—and adds only its own rules. It needed no change to the vision pipeline at all.

C. Toward More Games

Two games was never the end goal; it was the proof that the platform carries more than one. Because every game speaks only to the pointer snapshot and the dwell and gesture helpers, the genres that fit are the ones a single, occasionally vanishing, slightly jittery point can drive. Dwell-to-place tower defence is a clean fit: hold the light on a spot to build. So are whack-a-mole-style sweep games, light-painting and tracing, simple rhythm games where the laser hits targets on the beat, and hidden-object or reveal games that reuse the flashlight directly. What does *not* fit—typing, button-heavy menus, anything needing pixel-perfect clicks or two cursors—is ruled out by the same input layer that makes the good cases easy, which is the whole reason we built that layer narrow. Adding one of these is, in engineering terms, writing a new game module against an interface we already have, not extending the pipeline.

VI. Implementation and Deployment Challenges

A running theme of the project is that the hard parts of a live demo were in deployment, not in the algorithms. We describe the three that shaped the system most, each as problem, diagnosis, and fix.

Camera pipeline fragility. Getting the phone's camera into the laptop as a virtual webcam was the single biggest time sink. With one popular phone-as-webcam bridge, OpenCV would variously throw a backend exception, open the wrong physical device (including the laptop's infrared Windows-Hello camera), or hand back a single frozen frame. We wrote a small diagnostic that reads many frames in a row and reports whether the mean brightness *changes* (a live stream) or stays *constant* (a dead buffer), along with the negotiated pixel format; it showed the feed was a static buffer with no real stream negotiated, which we confirmed when an external capture tool reading the same virtual device also got a frozen frame while the bridge's own window showed live video. We switched to a phone-as-webcam application that produced a genuine live device, and we specify a custom push-based camera app as a sturdier replacement in future work.

TABLE II
Detector Accuracy (Single-Class Green Laser, Nano YOLO)

Input	Split	mAP50	mAP50-95	P	R
640 (native)	test	0.881	0.429	0.851	0.847
640 (native)	val	0.951	0.483	0.919	0.931
320 (deployed)	**test**	**0.816**	**0.395**	**0.845**	**0.831**
320 (deployed)	val	0.858	0.374	0.857	0.827

Calibration through projected feedback. When the laptop screen is mirrored to the projector, any window that shows the camera feed—such as the calibration view—is itself projected and re-filmed, giving a recursive "infinity-mirror" image that makes clicking exact corners hard. We worked around it by clicking the outermost projected rectangle and by making the tuning and game tools never display the raw feed on the projected screen. The residual error from this is the main source of the small spotlight offset in Section VIII, which the saved constant offset largely cancels.

Making the reveal feel instant. On the GPU-less laptop the reveal could lag visibly behind the laser. Why is the subject of Section VII; the short version is that the fix was not one faster model but a sequence of changes—shrinking the inference input, removing per-frame surface allocations in the renderer, capping the render and detection rates so the two threads stop fighting for the CPU, and finally moving inference onto the integrated GPU.

VII. Experiments and Results

We report three things: how accurately the detector finds the dot, how well the learned thief plays in simulation, and how the full pipeline behaves in real time on the demonstration laptop. The last is the paper's main empirical contribution.

Detection accuracy. Table II reports the fine-tuned detector on the held-out test and validation splits at two input sizes. At the model's native size the test mAP50 is 0.88, with precision and recall both near 0.85. At the smaller 320-pixel input we actually deploy—chosen because it roughly halves inference cost on the laptop—test mAP50 falls to about 0.82. That gap is the accuracy we trade for speed, and on this game it is a trade worth making: a slightly looser box on a laser dot still gives a good centre, and the extra frames per second matter far more to how the reveal feels. Qualitatively, detection was reliable toward the left and right of the frame and weaker in the centre, which is what motivated the edge-biased maze (Section IV-C).

Learned thief in simulation. Trained for 20,000 episodes against the four scripted laser behaviours, the Q-learning thief reaches the treasure in 95 percent of simulated rounds and is caught in the other 5 percent. This is a simulation result against scripted opponents, not a measurement against human players, and we read it only as evidence that the learned policy navigates and evades competently on its fixed maze.

TABLE III
REAL-TIME BEHAVIOUR BY INFERENCE BACKEND ON THE GPU-LESS DEMONSTRATION LAPTOP (INTEL CORE I7, IRIS XE IGPU), INFERENCE INPUT SIZE 320

Configuration	Inference (fps)	Effect on real-time feel
PyTorch, CPU	∼9–10	Smoothest of the CPU-only options
PyTorch, CPU, naive full-screen render	∼3	Rendering and inference contend for the CPU
ONNX Runtime, CPU	∼14–17	Faster, but uncapped inference starves the renderer (screen ∼1 fps) until detection is rate-capped
OpenVINO, Iris Xe iGPU	**near-instant**	**Inference offloaded off the CPU; contention removed; smooth**

Real-time deployment: the inference-versus-render contention. Making the reveal feel instant on the GPU-less laptop, not improving accuracy, was the dominant challenge, and it produced the paper's main finding. Table III reports the observed inference rate and the resulting feel for four configurations at the deployed 320-pixel input.

The pattern in Table III is the result we want to stress. Plain PyTorch on the CPU was fine on its own, but the moment the full-screen renderer was added naively—reallocating a large overlay surface and rescaling the background every frame—inference collapsed to roughly three frames per second. The renderer was eating the CPU that inference needed. Switching the detector to ONNX Runtime made inference *faster* in isolation, about 14 to 17 frames per second, and yet the system felt worse, because the now-faster inference thread ran flat out and starved the renderer until we capped the detection rate to leave it idle gaps. So on this machine raw inference speed was not the binding constraint; contention for the one CPU between inference and rendering was. The configuration that broke the deadlock was OpenVINO targeting the Intel Iris Xe integrated GPU. By moving inference off the CPU entirely it left the processor free to render, removed the contention, and gave the near-instant reveal we were after—the biggest single gain of the project. Two render-side changes were needed alongside it: reusing one overlay surface and pre-scaling the background once instead of per frame, and capping the render and detection rates so neither thread monopolises the CPU. We note that the performance figures here are observations from run logs on a single laptop, not a controlled benchmark; we treat them as establishing the qualitative pattern, which was consistent across sessions, rather than precise constants.

VIII. DISCUSSION AND LIMITATIONS

Thermal throttling. The integrated-GPU configuration was near-instant, but after roughly a minute of sustained use its responsiveness drifted back toward the CPU baseline. We put this down to the shared power and thermal budget of Intel laptop processors: the CPU and the integrated GPU draw from one budget, so sustained inference heats the package and throttles both. Capping the detection rate (still instant to the eye, but less heat), running on wall power in a high-performance profile, and adding active cooling are the mitigations we identify.

Calibration sensitivity and spotlight offset. A single global four-corner homography cannot model lens distortion [14], and the infinity-mirror calibration (Section VI) makes exact corner selection harder. Together they leave a small, position-dependent offset between the laser and the centre of the revealed spotlight. A constant offset correction removes most of it, but an offset that grows toward the edges points to perspective or lens error that only better calibration can fix.

Dataset bias. The centre-versus-edge asymmetry is a property of our training data, not of the architecture. We turned it into a level-design rule with the edge-biased maze, but the principled fix is to retrain on footage shot under the real demonstration conditions, deliberately including centre-frame and varied-background cases.

Scope of the controllers and the games. Our 95 percent figure for the learned thief is against scripted lasers in simulation, not against people, and we have not run a formal user study of either game. The A* controller, while it generalises across mazes, only avoids the light as well as our hand-written costs tell it to. Both are honest, working controllers; neither has been evaluated against human players yet, and that evaluation is the obvious next step.

IX. CONCLUSION AND FUTURE WORK

We built LaserLearn, a real-time platform that carries a single green laser pointer from a phone camera, through a fine-tuned single-class detector and a four-corner homography, into a small laser-faithful input layer that more than one game can sit on. On that layer we built two games—a flashlight-reveal thief hunt with two interchangeable adversary controllers, and a laser-slicing arcade game—and the second cost almost nothing precisely because the platform was designed for it. Putting the system on a commodity, GPU-less laptop surfaced our main finding: the limit on real-time feel was not raw inference speed but contention for the processor between inference and rendering, and offloading inference to the integrated GPU through OpenVINO removed it and was the largest single improvement we made.

The clearest next step is more games. The platform was built to host them, and dwell-to-place tower defence, sweep and reveal games, and simple rhythm games all map onto the input layer we already have. Beyond that we want to put the two thief controllers in front of real players and measure them properly; retrain the detector on footage captured under true demonstration conditions to close the centre-frame gap; replace the fragile virtual-webcam bridge with a robust push-based phone-camera app; and explore integer quantization and active cooling to hold the integrated-GPU speed-up past the onset of thermal throttling.

REFERENCES

[1] C. Kirstein and H. Müller, "Interaction with a projection screen using a camera-tracked laser pointer," in *Proc. Int. Conf. Multimedia Modeling (MMM)*, Lausanne, Switzerland, 1998, pp. 191–192.

[2] D. R. Olsen Jr. and T. Nielsen, "Laser pointer interaction," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, Seattle, WA, USA, 2001, pp. 17–22.

[3] B. A. Ahlborn, D. Thompson, O. Kreylos, B. Hamann, and O. G. Staadt, "A practical system for laser pointer interaction on large displays," in *Proc. ACM Symp. Virtual Reality Software and Technology (VRST)*, Monterey, CA, USA, 2005, pp. 106–109.

[4] J.-F. Lapointe and G. Godin, "On-screen laser spot detection for large display interaction," in *Proc. IEEE Int. Workshop Haptic Audio Visual Environments and their Applications (HAVE)*, Ottawa, ON, Canada, 2005, pp. 72–76.

[5] D. Krstinić, A. Kuzmanić Skelin, and I. Milatić, "Laser spot tracking based on modified circular Hough transform and motion pattern analysis," *Sensors*, vol. 14, no. 11, pp. 20112–20133, 2014.

[6] F. Chávez, F. Fernández, M. J. Gacto, and R. Alcalá, "Hybrid laser pointer detection algorithm based on template matching and fuzzy rule-based systems for domotic control in real home environments," *Applied Intelligence*, vol. 36, no. 4, pp. 907–926, 2012.

[7] R. Raskar, G. Welch, M. Cutts, A. Lake, L. Stesin, and H. Fuchs, "The office of the future: A unified approach to image-based modeling and spatially immersive displays," in *Proc. SIGGRAPH*, Orlando, FL, USA, 1998, pp. 179–188.

[8] R. Raskar, G. Welch, and H. Fuchs, "Spatially augmented reality," in *Proc. 1st IEEE Int. Workshop Augmented Reality (IWAR)*, San Francisco, CA, USA, 1998.

[9] R. Raskar, G. Welch, K.-L. Low, and D. Bandyopadhyay, "Shader lamps: Animating real objects with image-based illumination," in *Proc. 12th Eurographics Workshop on Rendering (EGWR)*, London, U.K., 2001, pp. 89–102.

[10] C. Pinhanez, "The everywhere displays projector: A device to create ubiquitous graphical interfaces," in *Proc. Int. Conf. Ubiquitous Computing (UbiComp)*, LNCS 2201, Atlanta, GA, USA, 2001, pp. 315–331.

[11] B. R. Jones, H. Benko, E. Ofek, and A. D. Wilson, "IllumiRoom: Peripheral projected illusions for interactive experiences," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI)*, Paris, France, 2013, pp. 869–878.

[12] B. Jones, R. Sodhi, M. Murdock, R. Mehra, H. Benko, A. Wilson, E. Ofek, B. MacIntyre, N. Raghuvanshi, and L. Shapira, "RoomAlive: Magical experiences enabled by scalable, adaptive projector-camera units," in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, Honolulu, HI, USA, 2014, pp. 637–644.

[13] P. Song, S. Winkler, and J. Tedjokusumo, "A tangible game interface using projector-camera systems," in *Proc. Int. Conf. Human-Computer Interaction (HCII)*, LNCS 4551, Beijing, China, 2007, pp. 956–965.

[14] D. Moreno and G. Taubin, "Simple, accurate, and robust projector-camera calibration," in *Proc. 2nd Int. Conf. 3D Imaging, Modeling, Processing, Visualization and Transmission (3DIMPVT)*, Zurich, Switzerland, 2012, pp. 464–471.

[15] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[16] B. Ridel, P. Reuter, J. Laviole, N. Mellado, N. Couture, and X. Granier, "The revealing flashlight: Interactive spatial augmented reality for detail exploration of cultural heritage artifacts," *ACM Journal on Computing and Cultural Heritage*, vol. 7, no. 2, art. 6, 2014.

[17] E. A. Bier, M. C. Stone, K. Pier, W. Buxton, and T. D. DeRose, "Toolglass and magic lenses: The see-through interface," in *Proc. SIGGRAPH*, Anaheim, CA, USA, 1993, pp. 73–80.

[18] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3–4, pp. 279–292, 1992.

[19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[20] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[21] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *Proc. 10th Int. Conf. Machine Learning (ICML)*, Amherst, MA, USA, 1993, pp. 330–337.

[22] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Proc. 11th Int. Conf. Machine Learning (ICML)*, New Brunswick, NJ, USA, 1994, pp. 157–163.

[23] C. de Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kulić, "Decentralized multi-agent pursuit using deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4552–4559, 2021.

[24] B. Baker, I. Kanitscheider, T. Markov, Y. Wu, G. Powell, B. McGrew, and I. Mordatch, "Emergent tool use from multi-agent autocurricula," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2020.

[25] F. G. Glavin and M. G. Madden, "Adaptive shooting for bots in first person shooter games using reinforcement learning," *IEEE Trans. Computational Intelligence and AI in Games*, vol. 7, no. 2, pp. 180–192, 2015.

[26] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[27] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[28] J. Terven, D.-M. Córdova-Esparza, and J.-A. Romero-González, "A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS," *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.

[29] Y. Gorbachev, M. Fedorov, I. Slavutin, A. Tugarev, M. Fatekhov, and Y. Tarkan, "OpenVINO deep learning workbench: Comprehensive analysis and tuning of neural networks inference," in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, Seoul, South Korea, 2019.

[30] "Comparative analysis of deep learning inference frameworks on edge devices," *Electronics*, vol. 14, no. 15, art. 2977, 2025. [Author list to be verified on publisher page.]

[31] "Multi-model running latency optimization in an edge computing paradigm," *Sensors*, vol. 22, no. 16, art. 6097, 2022. [Author list to be verified on publisher page.]